MANUAL PASO A PASO CON TDD

TANDA DOLOREÑA

Un sistema de ahorro rotativo vecinal, construido 100% con TDD

Desarrollo Móvil Integral — Proyecto complementario de práctica TDD

Aportaciones Turnos rotativos Animación al entregar el turno

1. ¿Qué vamos a construir?

Una tanda es una de las prácticas de ahorro comunitario más queridas de México: un grupo de vecinos aporta la misma cantidad de dinero en cada periodo (por ejemplo, cada semana), y en cada periodo UNA sola persona del grupo recibe el total reunido, siguiendo un orden de turnos ya acordado. El ciclo continúa hasta que todos los participantes han recibido su turno exactamente una vez.

Tanda Doloreña digitaliza esta práctica para un grupo de vecinos de Dolores Hidalgo: registra quién ya aportó en la ronda actual, decide cuándo se puede 'cerrar' la ronda y entregar el dinero, y muestra el progreso con animaciones — sin reemplazar la confianza entre vecinos, solo ayudando a llevar el control.

Reglas de negocio que vamos a proteger con pruebas

- Cada participante aporta como máximo una vez por ronda.
- Solo puede aportar quien realmente pertenece a la tanda.
- La ronda solo puede cerrarse (y entregar el turno) cuando TODOS los participantes ya aportaron.
- Al cerrar una ronda, el turno se entrega a la persona correspondiente según el orden acordado, y el sistema avanza automáticamente al siguiente turno.
- Una vez que todos los participantes ya recibieron su turno, la tanda termina y no se aceptan más aportaciones.

2. Estructura del proyecto

```
lib/
  modelos/
    participante.dart
    tanda.dart
    cierre_ronda.dart
  logica/
    resultado_aportacion.dart
    resultado_cierre.dart
    servicio_tanda.dart
  presentation/
    tanda_screen.dart          (interfaz animada, al final)
  main.dart
test/
  servicio_tanda_test.dart

flutter create tanda_dolorena
cd tanda_dolorena
```

> **BUENA PRÁCTICA —** Igual que en los dos proyectos anteriores (Lotería y Vota Dolores Hidalgo), toda la lógica de negocio vive en Dart puro, sin ningún import de Flutter. La animación es la última capa, sobre reglas ya probadas.

3. Modelos base

lib/modelos/participante.dart

```
class Participante {
  final String id;
  final String nombre;
  bool yaRecibioTurno;
  Participante({required this.id, required this.nombre, this.yaRecibioTurno = false});
}
```

lib/modelos/tanda.dart

```
import 'participante.dart';

class Tanda {
  final List<Participante> participantes;
  final List<String> ordenTurnos;
  final double montoAportacion;
  int turnoActual = 0;
  final Set<String> aportacionesRondaActual = {};

  Tanda({
    required this.participantes,
    required this.ordenTurnos,
    required this.montoAportacion,
  });
}
```

lib/modelos/cierre_ronda.dart

```
import 'participante.dart';
import '../logica/resultado_cierre.dart';

class CierreRonda {
  final ResultadoCierre resultado;
  final Participante? beneficiario;
  CierreRonda(this.resultado, this.beneficiario);
}
```

lib/logica/resultado_aportacion.dart

```
enum ResultadoAportacion { registrada, yaAporto, participanteInvalido, tandaTerminada }
```

lib/logica/resultado_cierre.dart

```
enum ResultadoCierre { rondaCerrada, faltanAportaciones, tandaYaTermino }
```

Igual que en el proyecto de votación, preferimos valores de resultado explícitos (enums) en lugar de excepciones para los casos esperados del negocio — así el código que llama a estos métodos puede decidir con claridad qué mostrarle a los vecinos.

RONDA 1 — Registrar una aportación válida

ROJO — Escribe una prueba que falle

test/servicio_tanda_test.dart

```
import 'package:flutter_test/flutter_test.dart';
import 'package:tanda_dolorena/modelos/participante.dart';
import 'package:tanda_dolorena/modelos/tanda.dart';
import 'package:tanda_dolorena/logica/resultado_aportacion.dart';
import 'package:tanda_dolorena/logica/resultado_cierre.dart';
import 'package:tanda_dolorena/logica/servicio_tanda.dart';

Tanda _crearTandaDePrueba() {
  return Tanda(
    participantes: [
      Participante(id: 'p1', nombre: 'Ana'),
      Participante(id: 'p2', nombre: 'Beto'),
      Participante(id: 'p3', nombre: 'Carla'),
    ],
    ordenTurnos: ['p1', 'p2', 'p3'],
    montoAportacion: 200,
  );
}

void main() {
  test('registrar una aportacion valida la agrega a la ronda actual', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);

    final resultado = servicio.registrarAportacion('p1');

    expect(resultado, ResultadoAportacion.registrada);
    expect(tanda.aportacionesRondaActual.contains('p1'), true);
  });
}
```

VERDE — El mínimo código para pasar

lib/logica/servicio_tanda.dart

```
import '../modelos/tanda.dart';
import '../modelos/cierre_ronda.dart';
import 'resultado_aportacion.dart';
import 'resultado_cierre.dart';

class ServicioTanda {
  final Tanda tanda;
  ServicioTanda(this.tanda);

  ResultadoAportacion registrarAportacion(String idParticipante) {
    tanda.aportacionesRondaActual.add(idParticipante);
    return ResultadoAportacion.registrada;
  }
}
```

RONDA 2 — Rechazar una aportación repetida

ROJO — Escribe una prueba que falle

test/servicio_tanda_test.dart (prueba agregada)

```
  test('un participante no puede aportar dos veces en la misma ronda', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);
    servicio.registrarAportacion('p1');

    final segundoIntento = servicio.registrarAportacion('p1');

    expect(segundoIntento, ResultadoAportacion.yaAporto);
  });
```

VERDE — El mínimo código para pasar

lib/logica/servicio_tanda.dart (método corregido)

```
  ResultadoAportacion registrarAportacion(String idParticipante) {
    if (tanda.aportacionesRondaActual.contains(idParticipante)) {
      return ResultadoAportacion.yaAporto;
    }
    tanda.aportacionesRondaActual.add(idParticipante);
    return ResultadoAportacion.registrada;
  }
```

RONDA 3 — Rechazar aportaciones de quien no es participante

ROJO — Escribe una prueba que falle

test/servicio_tanda_test.dart (prueba agregada)

```
  test('alguien que no pertenece a la tanda no puede aportar', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);

    final resultado = servicio.registrarAportacion('desconocido');

    expect(resultado, ResultadoAportacion.participanteInvalido);
  });
```

VERDE — El mínimo código para pasar

lib/logica/servicio_tanda.dart (método corregido)

```
  ResultadoAportacion registrarAportacion(String idParticipante) {
    final esParticipante = tanda.participantes.any((p) => p.id == idParticipante);
    if (!esParticipante) return ResultadoAportacion.participanteInvalido;

    if (tanda.aportacionesRondaActual.contains(idParticipante)) {
      return ResultadoAportacion.yaAporto;
    }
    tanda.aportacionesRondaActual.add(idParticipante);
    return ResultadoAportacion.registrada;
```

```
}
```

RONDA 4 — Detectar cuándo todos ya aportaron

ROJO — Escribe una prueba que falle

test/servicio_tanda_test.dart (pruebas agregadas)

```
  test('puedeCerrarRonda es falso si falta algun participante por aportar', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);
    servicio.registrarAportacion('p1');
    servicio.registrarAportacion('p2');

    expect(servicio.puedeCerrarRonda(), false);
  });

  test('puedeCerrarRonda es verdadero cuando todos ya aportaron', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);
    servicio.registrarAportacion('p1');
    servicio.registrarAportacion('p2');
    servicio.registrarAportacion('p3');

    expect(servicio.puedeCerrarRonda(), true);
  });
```

VERDE — El mínimo código para pasar

lib/logica/servicio_tanda.dart (método agregado)

```
  bool puedeCerrarRonda() {
    return tanda.participantes.every((p) => tanda.aportacionesRondaActual.contains(p.id));
  }
```

RONDA 5 — Cerrar la ronda y entregar el turno

ROJO — Escribe una prueba que falle

test/servicio_tanda_test.dart (prueba agregada)

```
  test('cerrar la ronda entrega el turno al beneficiario y avanza', () {
    final tanda = _crearTandaDePrueba(); // orden: p1, p2, p3
    final servicio = ServicioTanda(tanda);
    servicio.registrarAportacion('p1');
    servicio.registrarAportacion('p2');
    servicio.registrarAportacion('p3');

    final cierre = servicio.cerrarRondaYAvanzar();

    expect(cierre.resultado, ResultadoCierre.rondaCerrada);
    expect(cierre.beneficiario!.id, 'p1');
    expect(tanda.participantes.firstWhere((p) => p.id == 'p1').yaRecibioTurno, true);
    expect(tanda.turnoActual, 1);
    expect(tanda.aportacionesRondaActual.isEmpty, true);
  });
```

VERDE — El mínimo código para pasar

lib/logica/servicio_tanda.dart (método agregado)

```
  CierreRonda cerrarRondaYAvanzar() {
    if (!puedeCerrarRonda()) {
      return CierreRonda(ResultadoCierre.faltanAportaciones, null);
    }

    final idBeneficiario = tanda.ordenTurnos[tanda.turnoActual];
    final beneficiario = tanda.participantes.firstWhere((p) => p.id == idBeneficiario);
    beneficiario.yaRecibioTurno = true;

    tanda.aportacionesRondaActual.clear();
    tanda.turnoActual++;

    return CierreRonda(ResultadoCierre.rondaCerrada, beneficiario);
  }
```

Nota que el beneficiario se determina por ordenTurnos[turnoActual], no por quién aportó primero ni por ningún criterio nuevo: la tanda ya tenía un orden acordado desde el principio, y este método simplemente lo respeta.

RONDA 6 — No cerrar la ronda si faltan aportaciones

ROJO — Escribe una prueba que falle

test/servicio_tanda_test.dart (prueba agregada)

```
  test('no se puede cerrar la ronda si algun participante no ha aportado', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);
    servicio.registrarAportacion('p1');
    servicio.registrarAportacion('p2'); // p3 no aporta

    final cierre = servicio.cerrarRondaYAvanzar();

    expect(cierre.resultado, ResultadoCierre.faltanAportaciones);
    expect(cierre.beneficiario, isNull);
    expect(tanda.turnoActual, 0); // no debio avanzar
  });
```

VERDE — El mínimo código para pasar

Esta prueba pasa de inmediato: el guard clause if (!puedeCerrarRonda()) de la Ronda 5 ya cubre exactamente este caso. No se necesita código nuevo — solo confirmamos, con una prueba dedicada, que ese comportamiento está protegido para siempre.

> **BUENA PRÁCTICA —** No descartes una prueba solo porque pasa sin escribir código nuevo. El valor de esta prueba no es 'encontrar un bug hoy' — es evitar que alguien, en seis meses, rompa esta regla sin darse cuenta.

RONDA 7 — Detectar cuándo la tanda ha terminado

ROJO — Escribe una prueba que falle

test/servicio_tanda_test.dart (pruebas agregadas)

```
  test('haTerminado es falso mientras queden turnos pendientes', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);
    expect(servicio.haTerminado(), false);
  });

  test('haTerminado es verdadero cuando todos ya recibieron su turno', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);
    for (final id in ['p1', 'p2', 'p3']) {
      servicio.registrarAportacion('p1');
      servicio.registrarAportacion('p2');
      servicio.registrarAportacion('p3');
      servicio.cerrarRondaYAvanzar();
    }
    expect(servicio.haTerminado(), true);
  });

  test('no se puede aportar una vez que la tanda ha terminado', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);
    for (final id in ['p1', 'p2', 'p3']) {
      servicio.registrarAportacion('p1');
      servicio.registrarAportacion('p2');
```

```
      servicio.registrarAportacion('p3');
      servicio.cerrarRondaYAvanzar();
    }

    final resultado = servicio.registrarAportacion('p1');

    expect(resultado, ResultadoAportacion.tandaTerminada);
  });
```

VERDE — El mínimo código para pasar

lib/logica/servicio_tanda.dart (métodos agregado y corregido)

```
  bool haTerminado() => tanda.turnoActual >= tanda.ordenTurnos.length;

  ResultadoAportacion registrarAportacion(String idParticipante) {
    if (haTerminado()) return ResultadoAportacion.tandaTerminada;

    final esParticipante = tanda.participantes.any((p) => p.id == idParticipante);
    if (!esParticipante) return ResultadoAportacion.participanteInvalido;

    if (tanda.aportacionesRondaActual.contains(idParticipante)) {
      return ResultadoAportacion.yaAporto;
    }
    tanda.aportacionesRondaActual.add(idParticipante);
    return ResultadoAportacion.registrada;
  }
```

RONDA 8 — Refactor: nombrar las condiciones como reglas de negocio

REFACTOR — Mejora el código sin romper nada

Con las 7 rondas anteriores en verde, damos nombre a cada condición de registrarAportacion(), para que el método se lea como una lista de reglas de negocio en español, igual que hicimos en el proyecto de votación.

lib/logica/servicio_tanda.dart (refactorizado)

```
  ResultadoAportacion registrarAportacion(String idParticipante) {
    if (haTerminado()) return ResultadoAportacion.tandaTerminada;

    final esParticipante = tanda.participantes.any((p) => p.id == idParticipante);
    if (!esParticipante) return ResultadoAportacion.participanteInvalido;

    final yaAporto = tanda.aportacionesRondaActual.contains(idParticipante);
    if (yaAporto) return ResultadoAportacion.yaAporto;

    tanda.aportacionesRondaActual.add(idParticipante);
    return ResultadoAportacion.registrada;
  }
```

Las 8 pruebas anteriores siguen pasando exactamente igual: este refactor es puramente de legibilidad.

4. Prueba de integración: una tanda completa de principio a fin

test/servicio_tanda_test.dart (prueba de integración)

```
  test('simulacion completa: 3 vecinas completan una tanda de principio a fin', () {
    final tanda = _crearTandaDePrueba();
    final servicio = ServicioTanda(tanda);
    final beneficiarios = <String>[];

    while (!servicio.haTerminado()) {
      for (final p in tanda.participantes) {
        servicio.registrarAportacion(p.id);
      }
      final cierre = servicio.cerrarRondaYAvanzar();
      beneficiarios.add(cierre.beneficiario!.id);
    }

    expect(beneficiarios, ['p1', 'p2', 'p3']);
    expect(tanda.participantes.every((p) => p.yaRecibioTurno), true);
    expect(servicio.registrarAportacion('p1'), ResultadoAportacion.tandaTerminada);
  });
```

Esta prueba confirma, de principio a fin, que el orden de turnos se respeta exactamente (p1, p2, p3, en ese orden), que todos terminan con yaRecibioTurno en true, y que la tanda se cierra correctamente al final.

5. La interfaz gráfica animada

La pantalla muestra el orden de turnos, resalta con una animación de pulso a quien recibirá el turno actual, un contador animado de cuántos ya aportaron en la ronda, y una revelación animada del beneficiario al cerrar cada ronda.

lib/presentation/tanda_screen.dart

```
import 'package:flutter/material.dart';
import '../modelos/participante.dart';
import '../modelos/tanda.dart';
import '../logica/resultado_cierre.dart';
import '../logica/servicio_tanda.dart';

class TandaScreen extends StatefulWidget {
  const TandaScreen({super.key});
  @override
  State<TandaScreen> createState() => _TandaScreenState();
}

class _TandaScreenState extends State<TandaScreen> {
  late final Tanda _tanda;
  late final ServicioTanda _servicio;

  @override
  void initState() {
    super.initState();
    _tanda = Tanda(
      participantes: [
        Participante(id: 'p1', nombre: 'Ana'),
        Participante(id: 'p2', nombre: 'Beto'),
        Participante(id: 'p3', nombre: 'Carla'),
        Participante(id: 'p4', nombre: 'Diego'),
      ],
      ordenTurnos: ['p1', 'p2', 'p3', 'p4'],
      montoAportacion: 200,
    );
    _servicio = ServicioTanda(_tanda);
  }

  void _aportar(String idParticipante) {
    setState(() => _servicio.registrarAportacion(idParticipante));
  }

  void _cerrarRonda() {
    final cierre = _servicio.cerrarRondaYAvanzar();
    if (cierre.resultado == ResultadoCierre.faltanAportaciones) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('Todavia faltan vecinos por aportar.')),
      );
      return;
    }
    setState(() {});
    showGeneralDialog(
      context: context,
      barrierDismissible: true,
      barrierLabel: 'beneficiario',
      transitionDuration: const Duration(milliseconds: 450),
      pageBuilder: (context, anim1, anim2) => const SizedBox.shrink(),
      transitionBuilder: (context, anim1, anim2, child) {
        return ScaleTransition(
```

```
          scale: CurvedAnimation(parent: anim1, curve: Curves.elasticOut),
          child: FadeTransition(
            opacity: anim1,
            child: AlertDialog(
              title: const Text('Turno entregado'),
              content: Text('Le toco a ' + cierre.beneficiario!.nombre + ' esta ronda!'),
              actions: [
                TextButton(onPressed: () => Navigator.pop(context), child: const Text('Cerrar')),
              ],
            ),
          ),
        );
      },
    );
  }
```

lib/presentation/tanda_screen.dart (continuación: build)

```
  @override
  Widget build(BuildContext context) {
    final totalAportado = _tanda.aportacionesRondaActual.length;
    final totalParticipantes = _tanda.participantes.length;

    return Scaffold(
      appBar: AppBar(title: const Text('Tanda Dolorena')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text('Ronda actual: ' + (_tanda.turnoActual + 1).toString() + ' de ' +
_tanda.ordenTurnos.length.toString(),
              style: const TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),
          const SizedBox(height: 8),
          TweenAnimationBuilder<double>(
            tween: Tween(begin: 0, end: totalParticipantes == 0 ? 0 : totalAportado /
totalParticipantes),
            duration: const Duration(milliseconds: 500),
            curve: Curves.easeOutCubic,
            builder: (context, valor, _) => ClipRRect(
              borderRadius: BorderRadius.circular(8),
              child: LinearProgressIndicator(value: valor, minHeight: 10),
            ),
          ),
          const SizedBox(height: 4),
          Text(totalAportado.toString() + ' de ' + totalParticipantes.toString() + ' vecinos ya
aportaron'),
          const SizedBox(height: 20),
          ..._tanda.participantes.asMap().entries.map((entry) {
            final indice = entry.key;
            final participante = entry.value;
            final esTurnoActual = _tanda.ordenTurnos[_tanda.turnoActual] == participante.id;
            final yaAporto = _tanda.aportacionesRondaActual.contains(participante.id);

            return AnimatedContainer(
              duration: const Duration(milliseconds: 400),
              margin: const EdgeInsets.symmetric(vertical: 6),
              padding: const EdgeInsets.all(12),
              decoration: BoxDecoration(
                color: esTurnoActual ? Colors.amber.shade100 : Colors.grey.shade100,
                border: Border.all(
                  color: esTurnoActual ? Colors.amber : Colors.grey.shade300,
                  width: esTurnoActual ? 2 : 1,
                ),
```

```
                borderRadius: BorderRadius.circular(12),
              ),
              child: Row(
                children: [
                  Icon(
                    participante.yaRecibioTurno ? Icons.check_circle : Icons.person,
                    color: participante.yaRecibioTurno ? Colors.green : Colors.indigo,
                  ),
                  const SizedBox(width: 10),
                  Expanded(child: Text(participante.nombre, style: const TextStyle(fontWeight:
FontWeight.w600))),
                  if (esTurnoActual)
                    const Padding(
                      padding: EdgeInsets.only(right: 8),
                      child: Text('Turno actual', style: TextStyle(color: Colors.deepOrange,
fontSize: 12)),
                    ),
                  OutlinedButton(
                    onPressed: yaAporto ? null : () => _aportar(participante.id),
                    child: Text(yaAporto ? 'Aporto' : 'Aportar'),
                  ),
                ],
              ),
            );
          }),
          const SizedBox(height: 20),
          ElevatedButton.icon(
            onPressed: _cerrarRonda,
            icon: const Icon(Icons.emoji_events),
            label: const Text('Cerrar ronda y entregar turno'),
          ),
        ],
      ),
    );
  }
}
```

> **BUENA PRÁCTICA —** El widget AnimatedContainer anima automáticamente los cambios de color y borde cuando esTurnoActual cambia entre rondas — no escribimos ninguna animación manual para eso, Flutter la calcula sola al detectar que las propiedades del contenedor cambiaron.

lib/main.dart

```
import 'package:flutter/material.dart';
import 'presentation/tanda_screen.dart';

void main() => runApp(const TandaApp());

class TandaApp extends StatelessWidget {
  const TandaApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Tanda Dolorena',
      theme: ThemeData(primarySwatch: Colors.indigo, useMaterial3: true),
      home: const TandaScreen(),
    );
  }
}
```

6. Cómo ejecutar todo

1. Ejecuta flutter test — deberían pasar 10 pruebas en verde (9 de las rondas + 1 de integración).
2. Ejecuta flutter run.
3. Toca 'Aportar' en cada vecino hasta completar la ronda.
4. Presiona 'Cerrar ronda y entregar turno' y observa la animación de revelación del beneficiario.
5. Repite hasta que los 4 vecinos hayan recibido su turno.

7. Checklist de este proyecto TDD

✓	Verificación
	Cada regla de negocio (aportación única, participante válido, cierre solo con todos aportando) tiene su propia prueba
	El orden de turnos se respeta exactamente como fue definido, no se reordena por quién aportó primero
	Existe una prueba que confirme que la tanda rechaza aportaciones después de terminar
	La interfaz (TandaScreen) no decide por sí misma quién recibe el turno — solo se lo pregunta a ServicioTanda
	La prueba de integración recorre una tanda completa y verifica el orden final de beneficiarios

8. Retos de extensión

> **RETO —** Agrega la posibilidad de que el orden de turnos se asigne al azar (barajado) al crear la tanda, y escribe una prueba que confirme que, sin importar el orden, cada participante recibe su turno exactamente una vez.

- Agrega un historial de rondas cerradas (fecha, beneficiario) que se pueda consultar más tarde.
- Permite que un participante 'falte' a una ronda y quede marcado para reponer su aportación en la siguiente, en lugar de bloquear el cierre por completo.
- Agrega notificaciones locales que avisen a los vecinos cuándo les toca aportar.
- Persiste el estado de la tanda con shared_preferences para que sobreviva a cerrar la app entre reuniones.

INSIGNIA DESBLOQUEADA: Cinturón Negro de Proyecto — construiste un sistema de ahorro rotativo con reglas de negocio reales, 100% guiado por pruebas